19. ROS2 Launch startup file configuration

1. Introduction to Launch

Until now, every time we run a ROS node, we have to open a new terminal and run a command. A robotic system has many nodes, and starting them like this every time is really troublesome. Is there a way to start all the nodes at once? The answer is of course yes, and that is the Launch startup file, which is a script used in the ROS system to start and configure multiple nodes.

In ROS2, launch is used to start multiple nodes and configure program runtime parameters. ROS2 launch files come in xml, yaml and python formats. This lesson uses the python format launch file as an example; compared with the other two formats, the python format is more flexible:

- Python has a great number of function libraries that can be used in a startup file;
- The general launch features and the specific launch features of ROS2 are written in Python, so launch features that XML and YAML may not expose are accessible;

The most important part of writing a ROS2 launch file in Python is to abstract each node, file, script, etc. into an action and start them through a unified interface.

Reference materials:

- launch system design document: ROS 2 launch system design document
- Official launch API documentation: launch API documentation (updated along with the official release)
- Preparation, create a function package to hold the program files

```
ros2 pkg create learn_launch --build-type ament_python
```

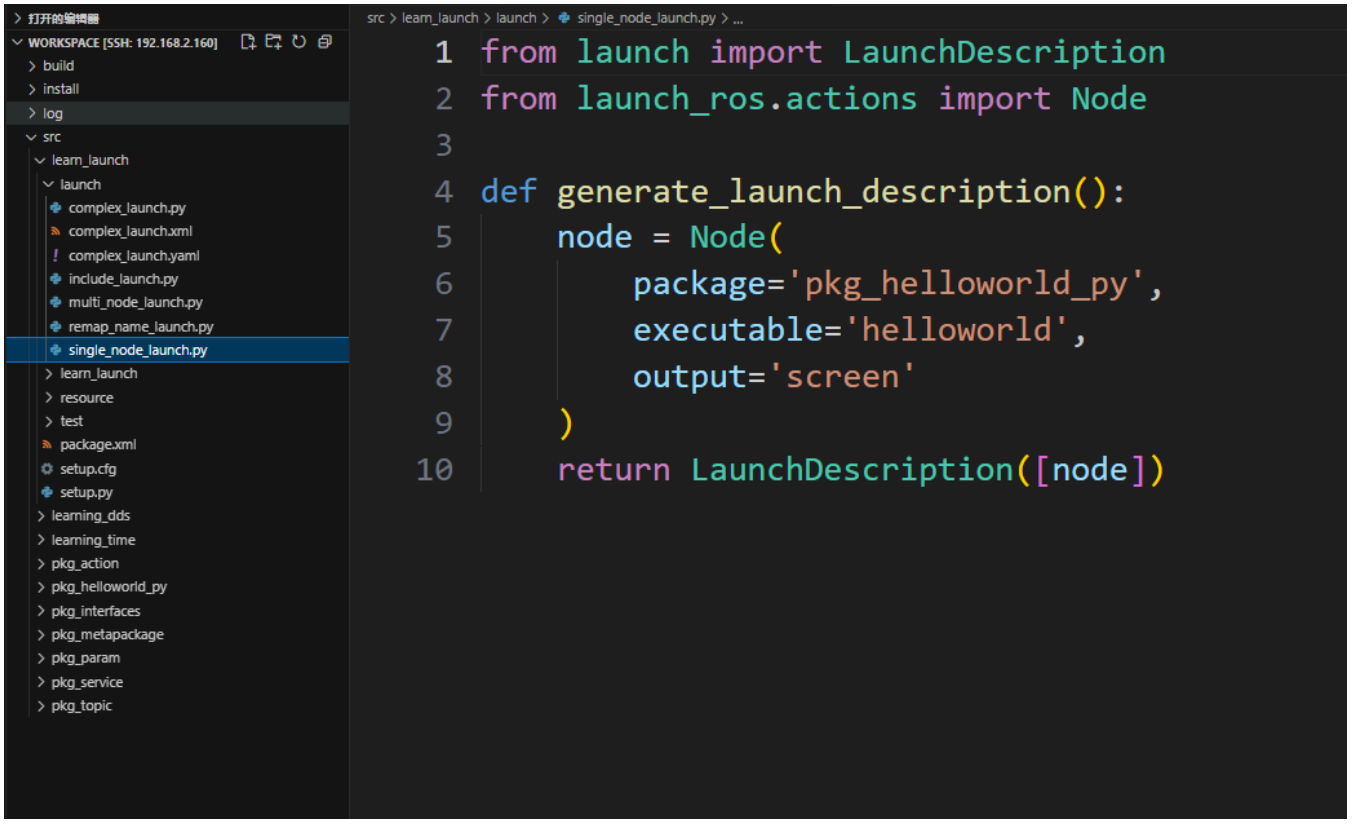
2. Writing the launch of a single Node

2.1. Creating a launch file

Create a launch folder under the function package, then create a file named [single_node_launch.py] inside the launch folder and copy the following content into that file:

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    node = Node(
        package='pkg_helloworld_py',
        executable='helloworld',
        output='screen'
    )
    return LaunchDescription([node])
```

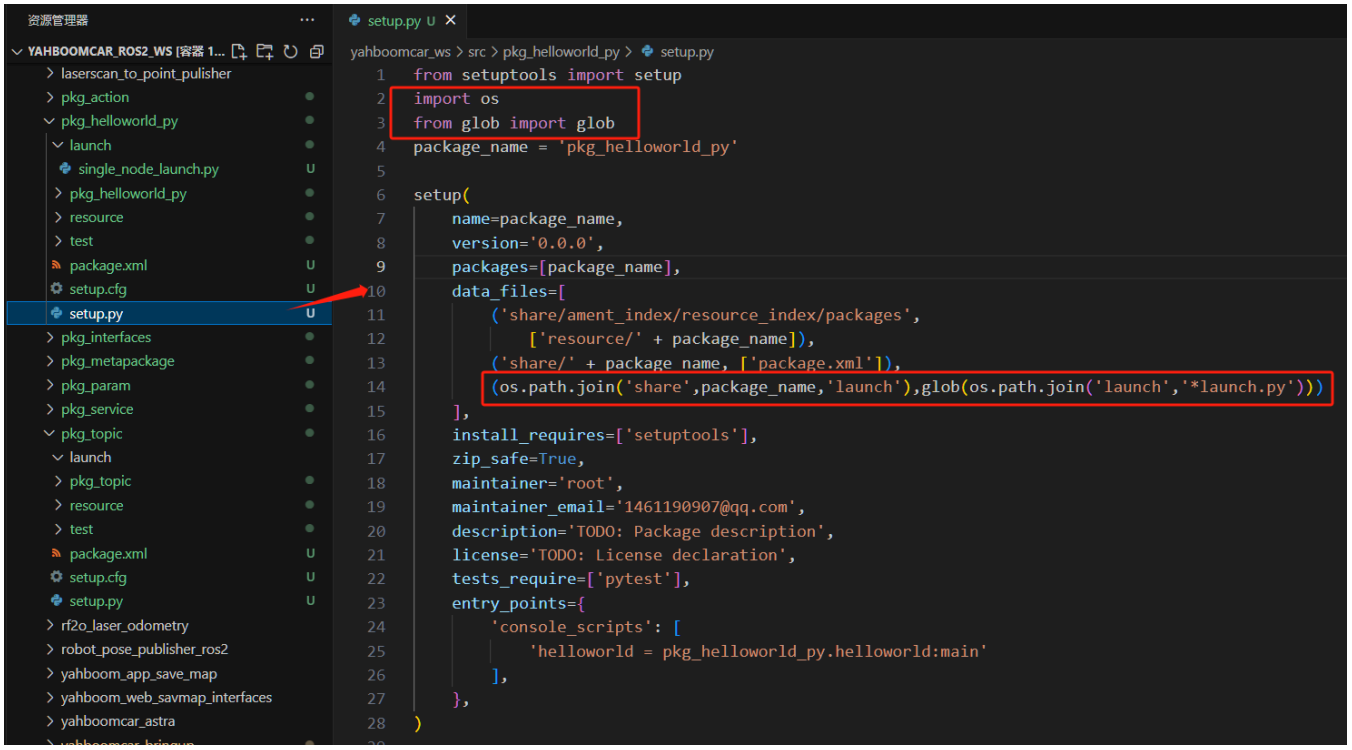


2.2. Configuring the setup.py file

Launch files are often named LaunchName_launch.py, where LaunchName is customizable and _launch.py is generally considered fixed. You need to modify the setup.py file under the function package, adding the files under the launch path, so that the executable .py file can be generated when compiling,

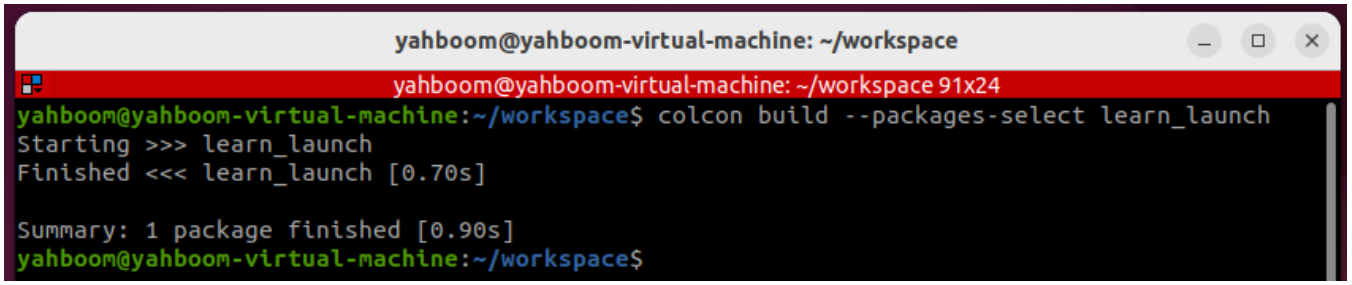
```
#1. Import the relevant header files
import os
from glob import glob

#2. In the data_files list, add the launch path and the launch.py file under that path
(os.path.join('share',package_name,'launch'),glob(os.path.join('launch','*launch.py')))
```



2.3. Compiling the function package

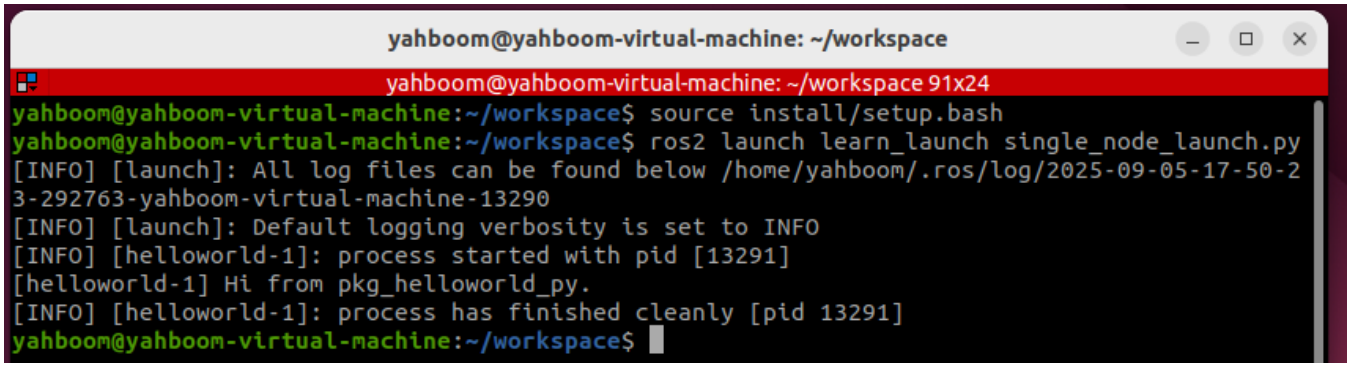
```
colcon build --packages-select learn_launch
```



2.4. Running the program

- Refresh the environment variables, then run the launch file

```
ros2 launch learn_launch single_node_launch.py
```



2.5. Source code analysis

1. Import the relevant libraries

```
from launch import LaunchDescription
from launch_ros.actions import Node
```

2. Define a function generate_launch_description and return a launch_description

```
def generate_launch_description():
    node = Node(
        package='pkg_helloworld_py',
        executable='helloworld',
    )
    return LaunchDescription([node])
```

A variable named node is defined as the return value of starting a node. The Node function is called, and the two important startup parameters, package and executable, are specified.

- package: indicates the function package, and represents the name of the function package.
- executable: indicates the program to be executed, the name of the executable program.

Finally, call the LaunchDescription function with the node parameter and return the result.

```
return LaunchDescription([node])
```

3. Writing the launch of multiple Nodes

3.1. Creating a launch file

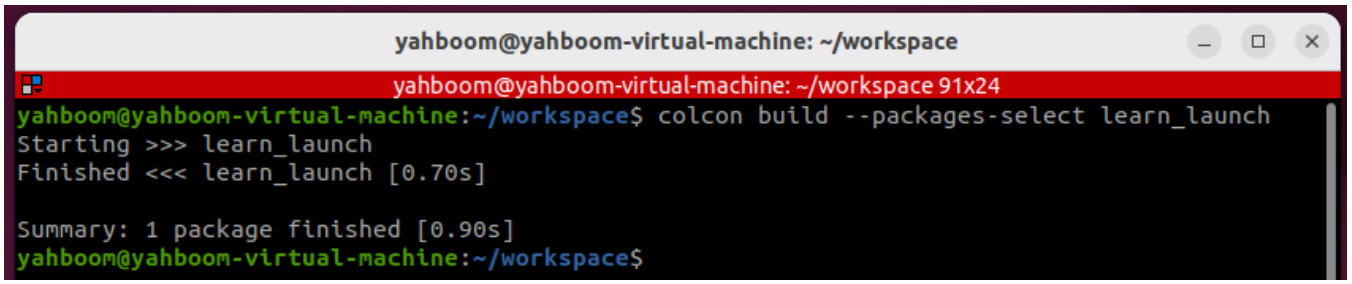
Create a file named [multi_node_launch.py] and add the following content:

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    publisher_node = Node(
        package='pkg_topic',
        executable='publisher_demo',
        output='screen'
    )
    subscriber_node = Node(
        package='pkg_topic',
        executable='subscriber_demo',
        output='screen'
    )
    return LaunchDescription([
        publisher_node,
        subscriber_node
    ])
```

3.2. Compile the function package

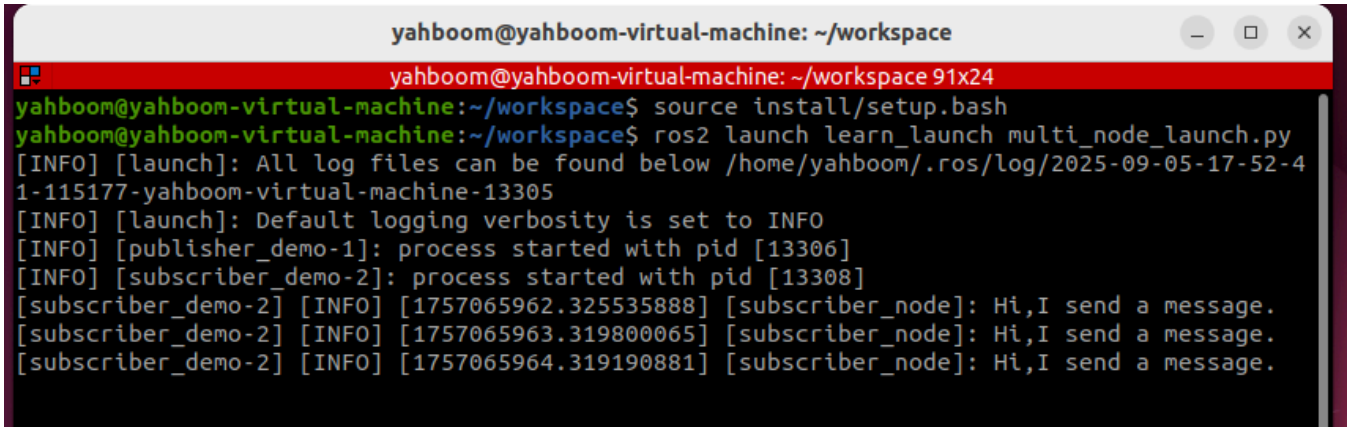
```
colcon build --packages-select learn_launch
```



3.2. Running the program

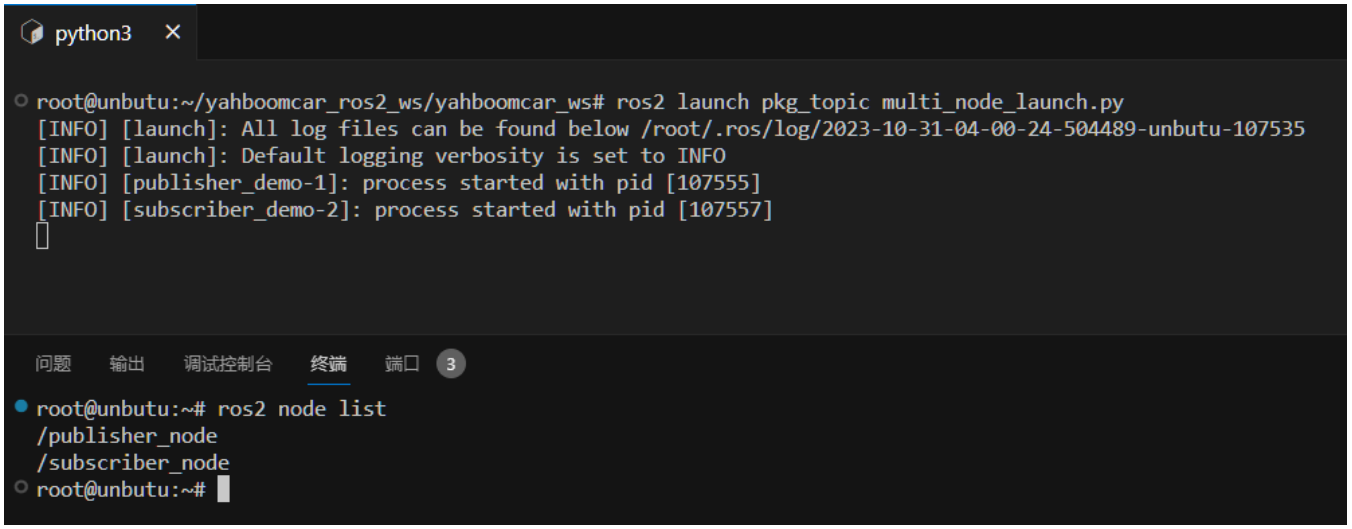
- Refresh the environment variables, then run the launch file

```
ros2 launch learn_launch multi_node_launch.py
```



If the terminal prints nothing, we can check which nodes have been started to verify whether they were launched successfully. Enter the following in the terminal:

```
ros2 node list
```



3.3. Source code analysis

It is roughly the same as simple_node_launch.py, except that there is one more node.

4. Topic remapping example

4.1. Creating a launch file

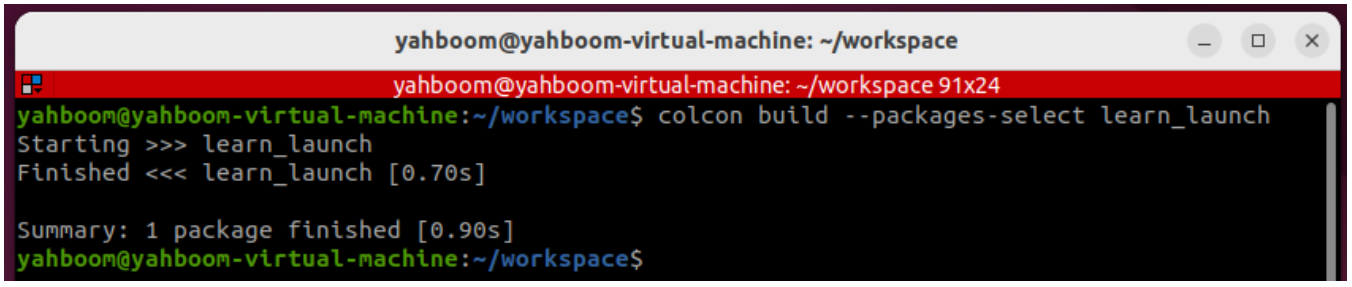
Create a file named [remap_name_launch.py] in the same directory as multi_node_launch.py and add the following content:

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    publisher_node = Node(
        package='pkg_topic',
        executable='publisher_demo',
        output='screen',
        remappings=[("/topic_demo", "/topic_update")]
    )
    return LaunchDescription([
        publisher_node
    ])
```

4.2. Compile the function package

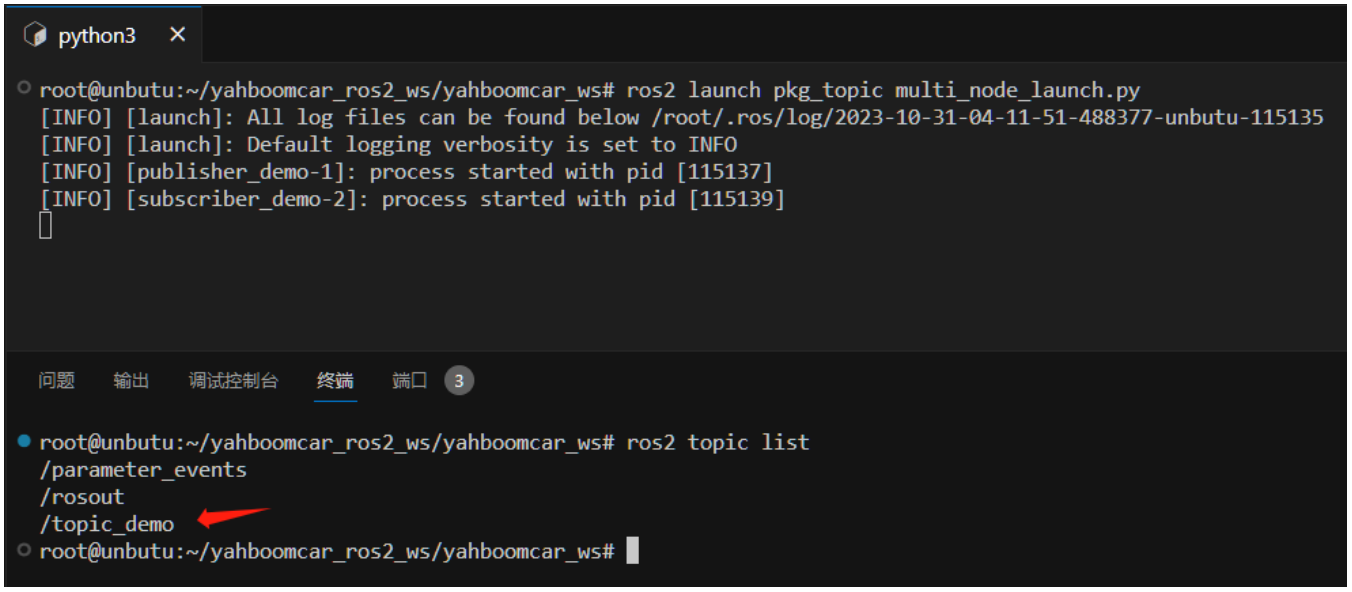
```
colcon build --packages-select learn_launch
```



4.2. Running the program

First, let's see what topic the publisher_demo node publishes before the topic is remapped:

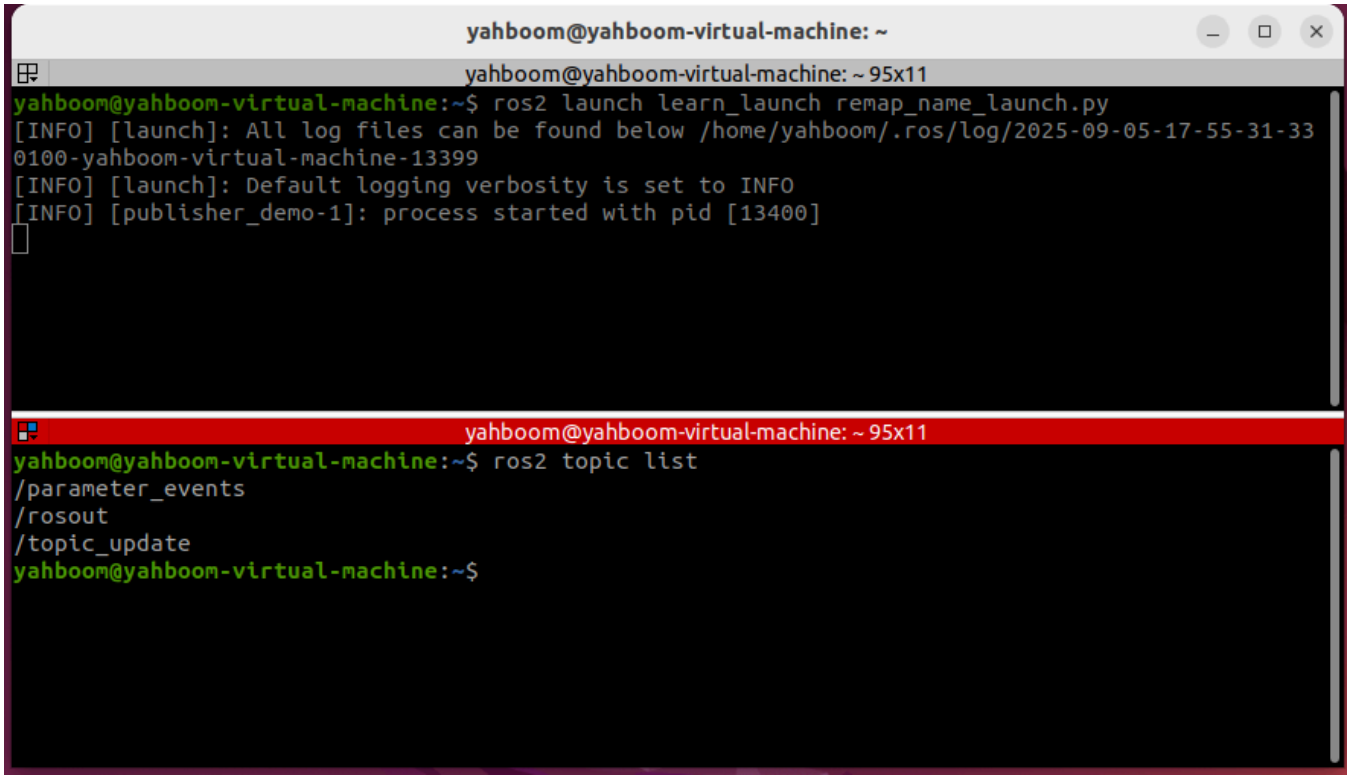
```
ros2 launch learn_launch multi_node_launch.py
ros2 topic list
```



The topic here is [/topic_demo]

- Refresh the environment variables again, run the program after the topic has been remapped, and see what changed:

```
ros2 launch learn_launch remap_name_launch.py
ros2 topic list
```



As shown in the figure above, the topic name has been remapped to [/topic_update]

4.3. Source code analysis

The main addition is the following part:

```
remappings=[("/topic_demo", "/topic_update")]
```

Here the original /topic_demo topic is remapped to /topic_update

5. Example of a launch file nested-starting another launch file

5.1. Creating a launch file

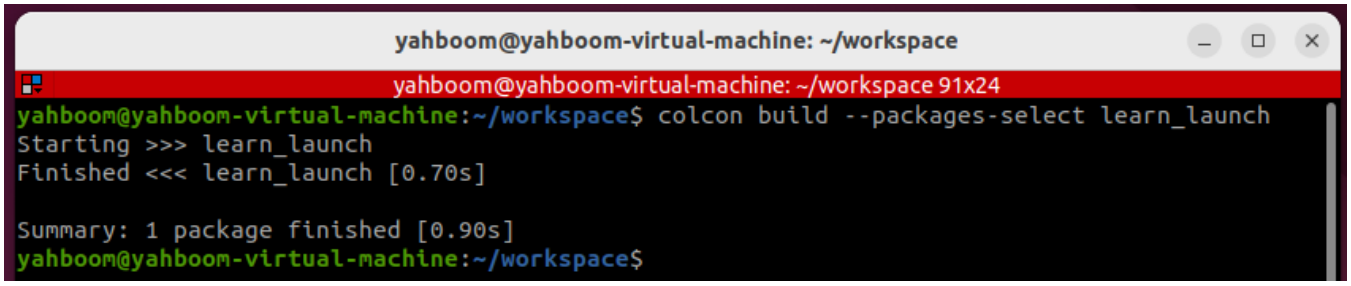
Create a file named [include_launch.py] in the same directory as multi_node_launch.py and add the following content:

```
from launch import LaunchDescription
import os
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    hello_launch = IncludeLaunchDescription(PythonLaunchDescriptionSource(
        os.path.join(
            get_package_share_directory('learn_launch'),
            'launch',
            'multi_node_launch.py'
        )),
    )
    return LaunchDescription([
        hello_launch
    ])
```

5.2. Compiling the function package

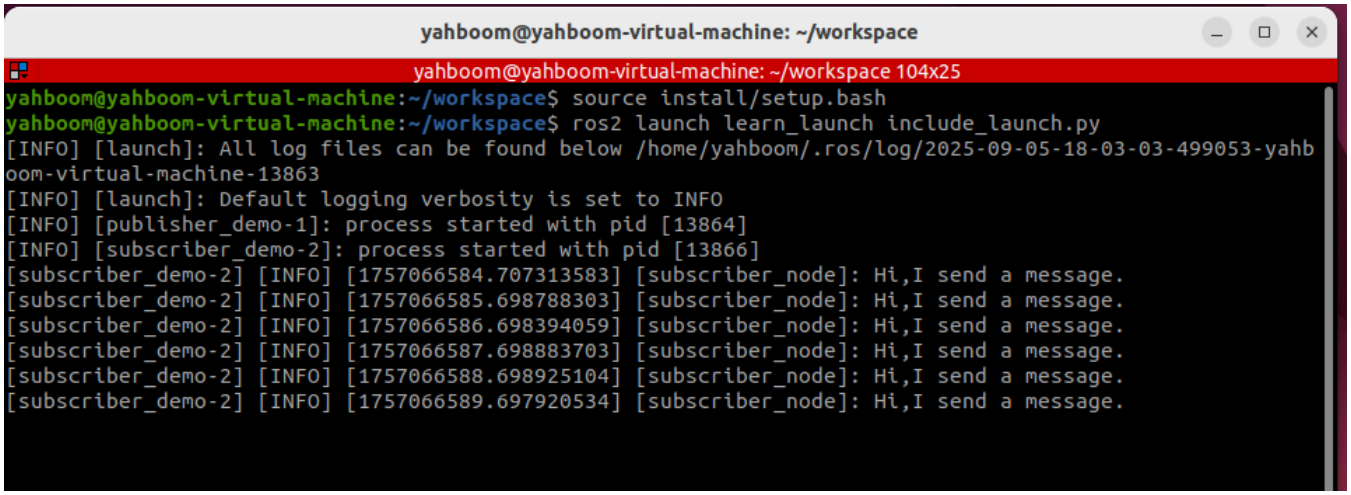
```
colcon build --packages-select learn_launch
```



5.3. Running the program

- Refresh the environment variables and run the launch file

```
ros2 launch learn_launch include_launch.py
```



5.4. Source code analysis

- Nested launch of a Python-format launch file requires the two classes `IncludeLaunchDescription` and `PythonLaunchDescriptionSource` of the launch system.
- `get_package_share_directory('learn_launch')`: obtains the shared directory of the function package, where `learn_launch` is the name of the function package;
- `os.path.join(..., 'launch', 'multi_node_launch.py')`: joins the relative path fragments to build the included Launch file. Do not put `/` in front of the file name, otherwise it will be treated as the root path of the file system.

6. Comprehensive launch file example

This example mainly shows how to write a complex launch file; the function of the program itself can be ignored.

6.1. Creating a launch file

Create a file named [complex_launch.py] in the same directory as multi_node_launch.py and add the following content:

```
import os
from ament_index_python import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.actions import IncludeLaunchDescription
from launch.actions import GroupAction
from launch.launch_description_sources import AnyLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration
from launch.substitutions import TextSubstitution
from launch_ros.actions import Node
```

```
from launch_ros.actions import PushRosNamespace


def generate_launch_description():

    # args that can be set from the command line or a default will be used
    background_r_launch_arg = DeclareLaunchArgument(
        "background_r", default_value=TextSubstitution(text="0")
    )
    background_g_launch_arg = DeclareLaunchArgument(
        "background_g", default_value=TextSubstitution(text="255")
    )
    background_b_launch_arg = DeclareLaunchArgument(
        "background_b", default_value=TextSubstitution(text="0")
    )
    chatter_ns_launch_arg = DeclareLaunchArgument(
        "chatter_ns", default_value=TextSubstitution(text="my/chatter/ns")
    )

    # include another launch file
    launch_include = IncludeLaunchDescription(
        AnyLaunchDescriptionSource(
            os.path.join(
                get_package_share_directory('demo_nodes_cpp'),
                'launch/topics/talker_listener_launch.xml'))
    )
    # include another launch file in the chatter_ns namespace
    launch_include_with_namespace = GroupAction(
        actions=[
            # push-ros-namespace to set namespace of included nodes
            PushRosNamespace(LaunchConfiguration('chatter_ns')),
            IncludeLaunchDescription(
                AnyLaunchDescriptionSource(
                    os.path.join(
                        get_package_share_directory('demo_nodes_cpp'),
                        'launch/topics/talker_listener_launch.xml'))
            ),
        ]
    )

    # start a turtlesim_node in the turtlesim1 namespace
    turtlesim_node = Node(
            package='turtlesim',
            namespace='turtlesim1',
            executable='turtlesim_node',
            name='sim'
        )

    # start another turtlesim_node in the turtlesim2 namespace
    # and use args to set parameters
    turtlesim_node_with_parameters = Node(
            package='turtlesim',
            namespace='turtlesim2',
```

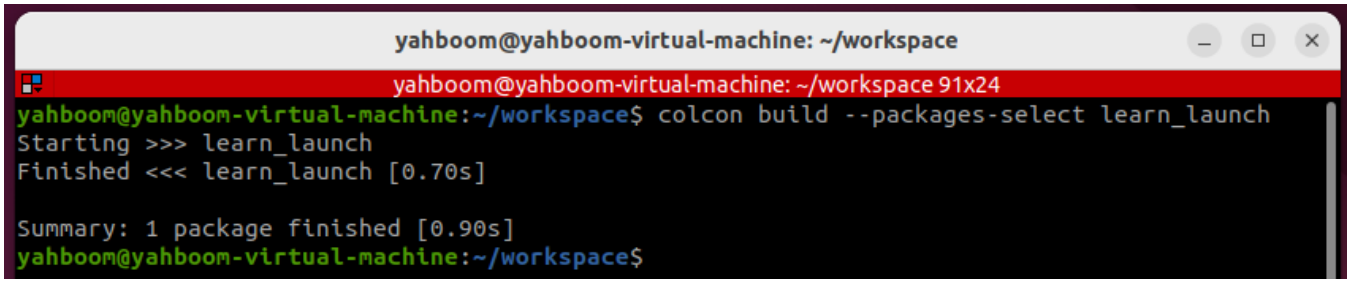
```
            executable='turtlesim_node',
            name='sim',
            parameters=[{
                "background_r": LaunchConfiguration('background_r'),
                "background_g": LaunchConfiguration('background_g'),
                "background_b": LaunchConfiguration('background_b'),
            }]
        )

    # perform remap so both turtles listen to the same command topic
    forward_turtlesim_commands_to_second_turtlesim_node = Node(
            package='turtlesim',
            executable='mimic',
            name='mimic',
            remappings=[
                ('/input/pose', '/turtlesim1/turtle1/pose'),
                ('/output/cmd_vel', '/turtlesim2/turtle1/cmd_vel'),
            ]
        )

    return LaunchDescription([
        background_r_launch_arg,
        background_g_launch_arg,
        background_b_launch_arg,
        chatter_ns_launch_arg,
        launch_include,
        launch_include_with_namespace,
        turtlesim_node,
        turtlesim_node_with_parameters,
        forward_turtlesim_commands_to_second_turtlesim_node,
    ])
```

6.2. Compile the workspace

```
colcon build --packages-select learn_launch
```

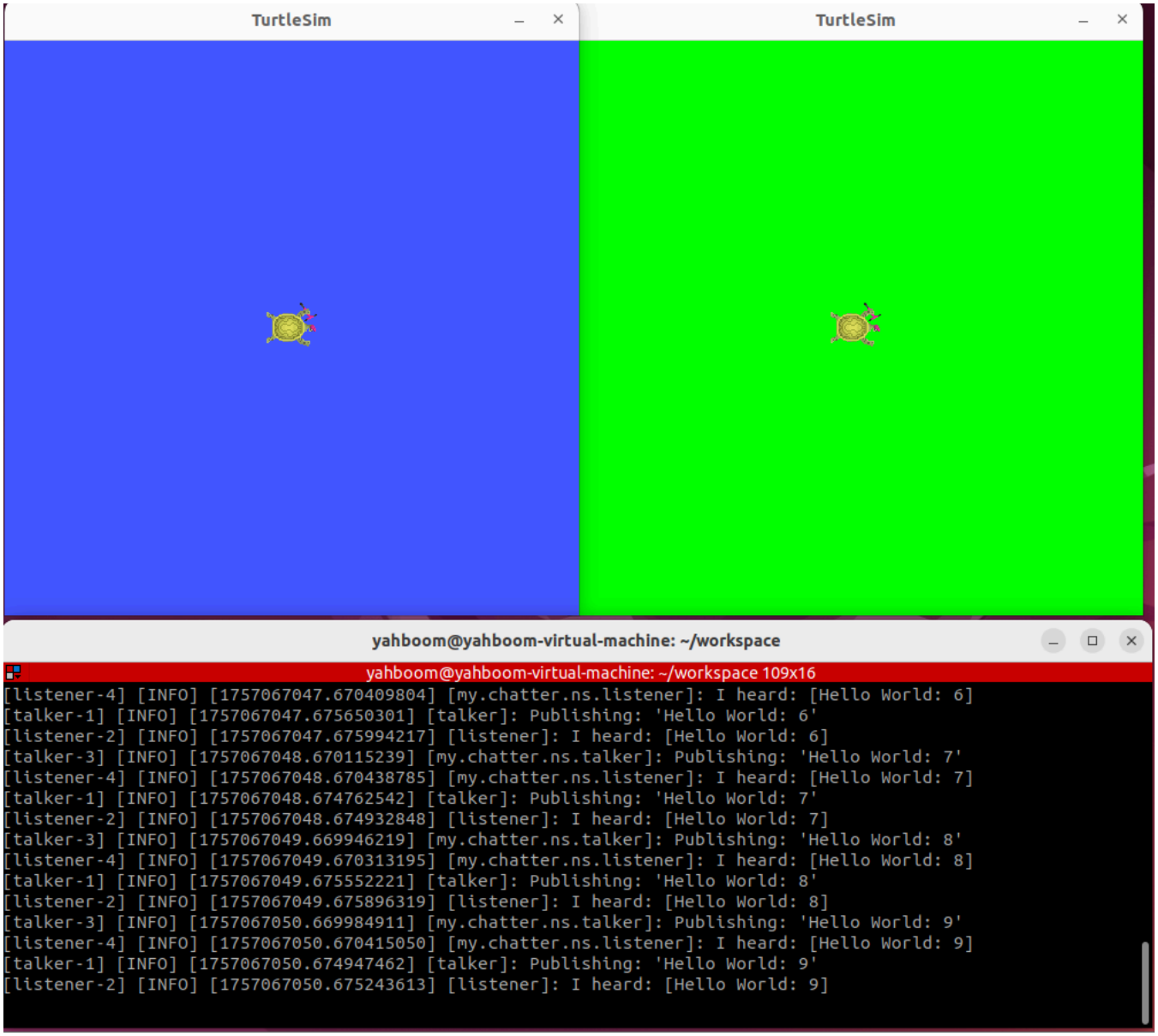


6.2. Running the program

- Refresh the environment variables in the terminal and run the launch file

```
ros2 launch learn_launch complex_launch.py
```

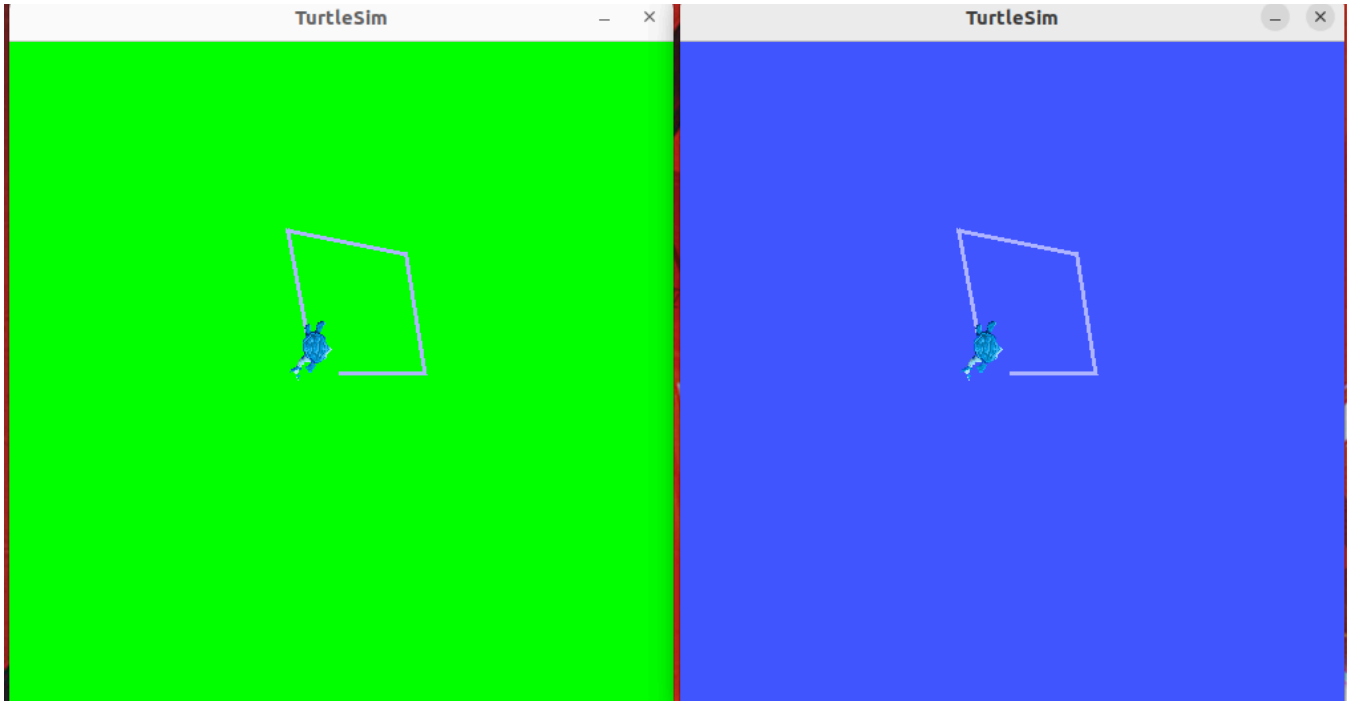
Two little turtles will be displayed in the VNC of the host machine.



- Start the keyboard control node and add a namespace (because we added a namespace when starting the node in the launch file)

```
ros2 run turtlesim turtle_teleop_key --ros-args -r __ns:=/turtlesim1
```

- Use the up, down, left and right keys to control the movement of turtle 1; turtle 2 will completely imitate the behavior of turtle 1



6.3. Program description

The program mainly starts:

1. The talker_listener node of demo_nodes_cpp,
2. The talker_listener node with a namespace
3. Little turtle 1, which uses turtlesim1 as its namespace
4. Little turtle 2, which uses turtlesim2 as its namespace
5. Performs the remapping so that both turtles can hear the same command topic

7. XML implementation

7.1. Creating a launch file

Create a file named [complex_launch.xml] in the same directory as complex_launch.py and add the following content:

```
<launch>

    <!-- args that can be set from the command line or a default will be used -->
    <arg name="background_r" default="0"/>
    <arg name="background_g" default="255"/>
    <arg name="background_b" default="0"/>
    <arg name="chatter_ns" default="my/chatter/ns"/>

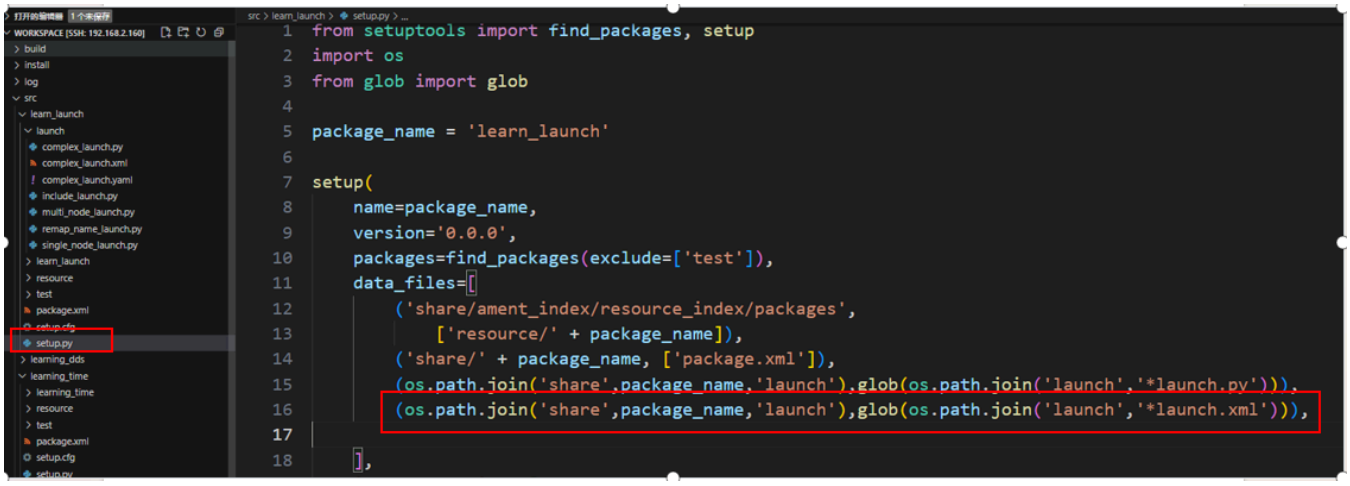
    <!-- include another launch file -->
    <include file="$(find-pkg-share
demo_nodes_cpp)/launch/topics/talker_listener_launch.xml"/>
    <!-- include another launch file in the chatter_ns namespace-->
    <group>
      <!-- push-ros-namespace to set namespace of included nodes -->
```

```
      <push-ros-namespace namespace="$(var chatter_ns)"/>
      <include file="$(find-pkg-share
demo_nodes_cpp)/launch/topics/talker_listener_launch.xml"/>
    </group>

    <!-- start a turtlesim_node in the turtlesim1 namespace -->
    <node pkg="turtlesim" exec="turtlesim_node" name="sim" namespace="turtlesim1"/>
    <!-- start another turtlesim_node in the turtlesim2 namespace
        and use args to set parameters -->
    <node pkg="turtlesim" exec="turtlesim_node" name="sim" namespace="turtlesim2">
      <param name="background_r" value="$(var background_r)"/>
      <param name="background_g" value="$(var background_g)"/>
      <param name="background_b" value="$(var background_b)"/>
    </node>
    <!-- perform remap so both turtles listen to the same command topic -->
    <node pkg="turtlesim" exec="mimic" name="mimic">
      <remap from="/input/pose" to="/turtlesim1/turtle1/pose"/>
      <remap from="/output/cmd_vel" to="/turtlesim2/turtle1/cmd_vel"/>
    </node>
</launch>
```

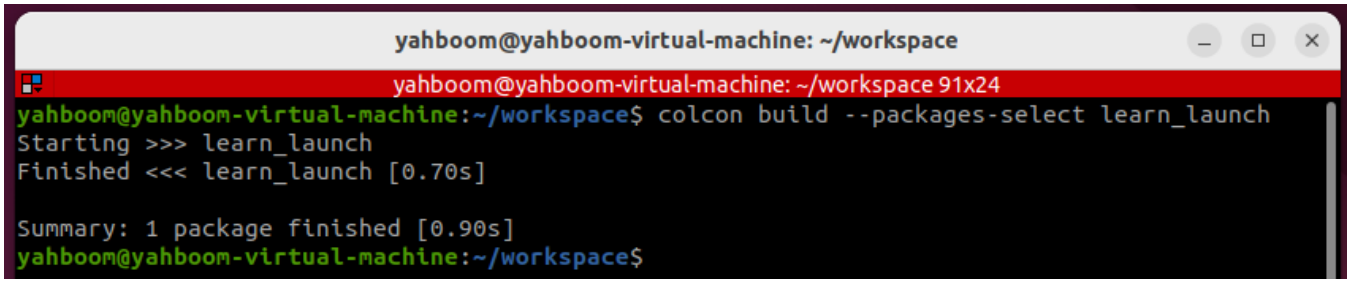
7.2. setup.py file configuration

- The compilation file needs to be configured so that our .xml format launch file is copied into the install directory at build time; only then can the ROS system find our file



7.3. Compiling the function package

```
colcon build --packages-select learn_launch
```

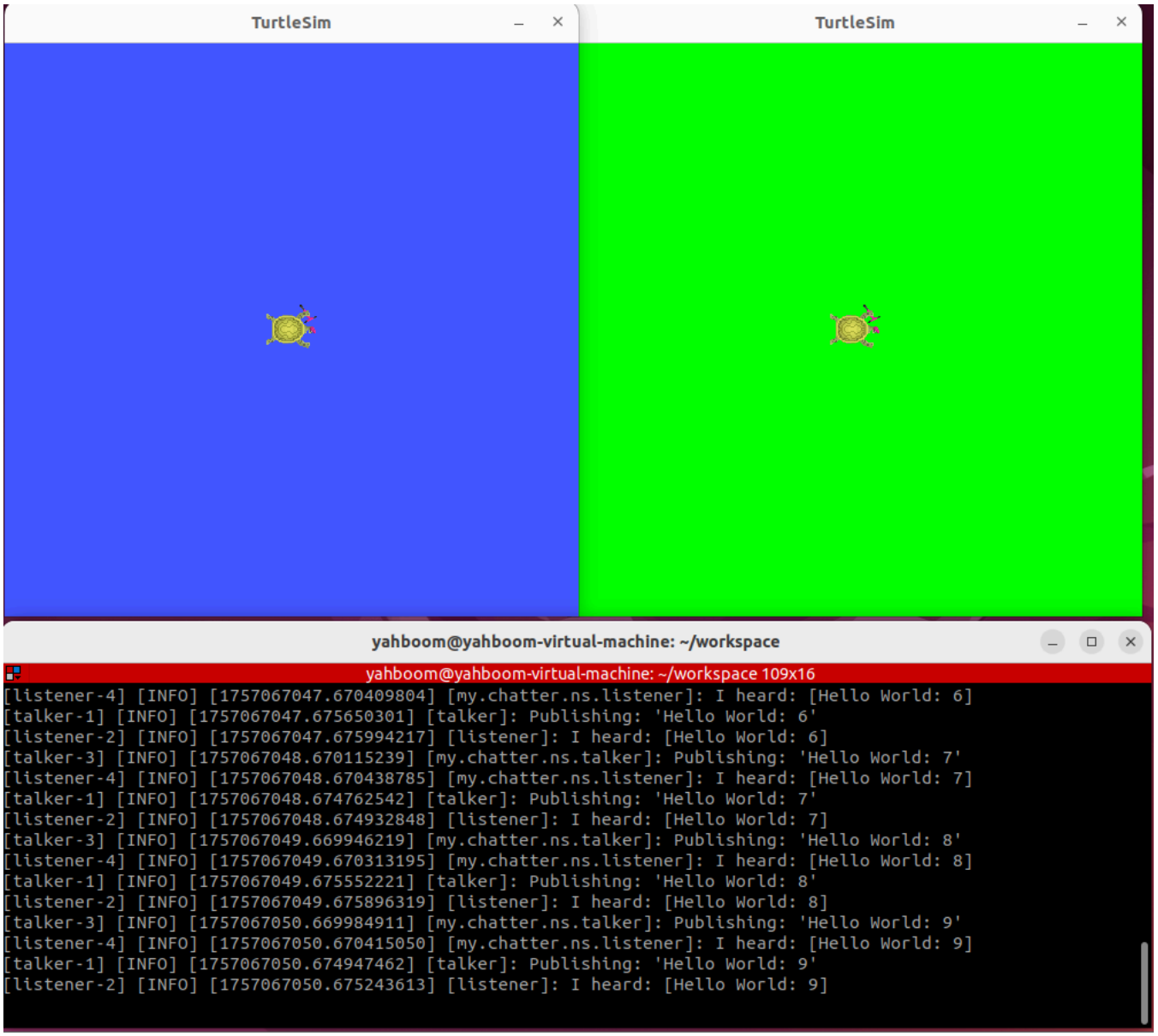


7.4. Running the program

Enter the following in the terminal:

```
ros2 launch learn_launch complex_launch.xml
```

- As expected, two little turtles will appear, and the terminal will print log information



- Start the keyboard control node and add a namespace

```
ros2 run turtlesim turtle_teleop_key --ros-args -r __ns:=/turtlesim1
```

Use the keyboard control to start turtle 1 running; turtle 2 will completely imitate the behavior of turtle 1

8. YAML implementation

8.1. Creating a launch file

Create a file named [complex_launch.yaml] in the same directory as complex_launch.py and add the following content:

```
launch:

# args that can be set from the command line or a default will be used
- arg:
    name: "background_r"
    default: "0"
- arg:
    name: "background_g"
    default: "255"
- arg:
    name: "background_b"
    default: "0"
- arg:
    name: "chatter_ns"
    default: "my/chatter/ns"


# include another launch file
- include:
    file: "$(find-pkg-share demo_nodes_cpp)/launch/topics/talker_listener_launch.xml"

# include another launch file in the chatter_ns namespace
- group:
    - push-ros-namespace:
        namespace: "$(var chatter_ns)"
    - include:
        file: "$(find-pkg-share demo_nodes_cpp)/launch/topics/talker_listener_launch.xml"

# start a turtlesim_node in the turtlesim1 namespace
- node:
    pkg: "turtlesim"
    exec: "turtlesim_node"
    name: "sim"
    namespace: "turtlesim1"

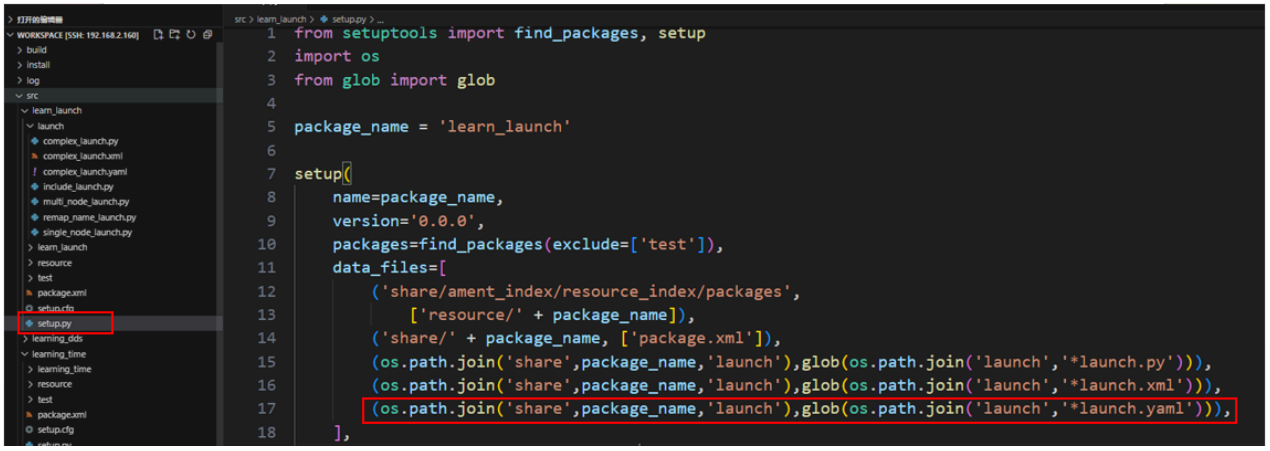
# start another turtlesim_node in the turtlesim2 namespace and use args to set parameters
- node:
    pkg: "turtlesim"
    exec: "turtlesim_node"
    name: "sim"
    namespace: "turtlesim2"
    param:
    -
      name: "background_r"
      value: "$(var background_r)"
```

```
    -
      name: "background_g"
      value: "$(var background_g)"
    -
      name: "background_b"
      value: "$(var background_b)"

# perform remap so both turtles listen to the same command topic
- node:
    pkg: "turtlesim"
    exec: "mimic"
    name: "mimic"
    remap:
    -
        from: "/input/pose"
        to: "/turtlesim1/turtle1/pose"
    -
        from: "/output/cmd_vel"
        to: "/turtlesim2/turtle1/cmd_vel"
```

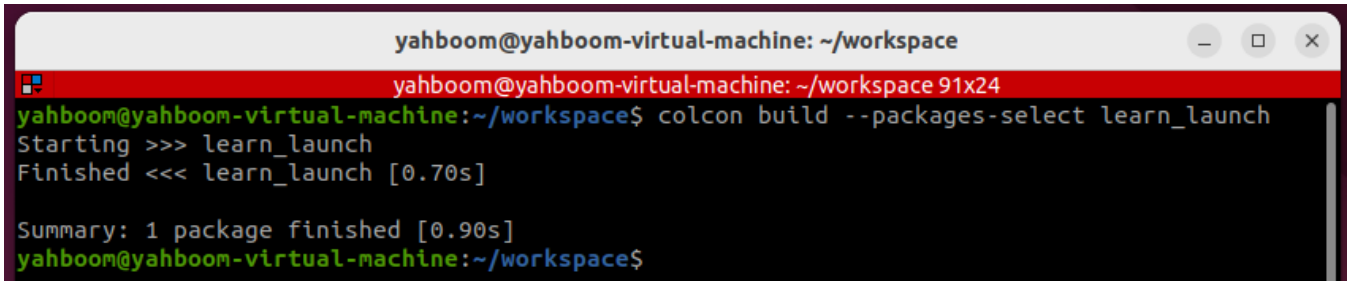
8.2. Configuration

- The compilation file needs to be configured so that our .yaml format launch file is copied into the install directory at build time; only then can the ROS system find our file



8.3. Compiling the function package

```
colcon build --packages-select learn_launch
```

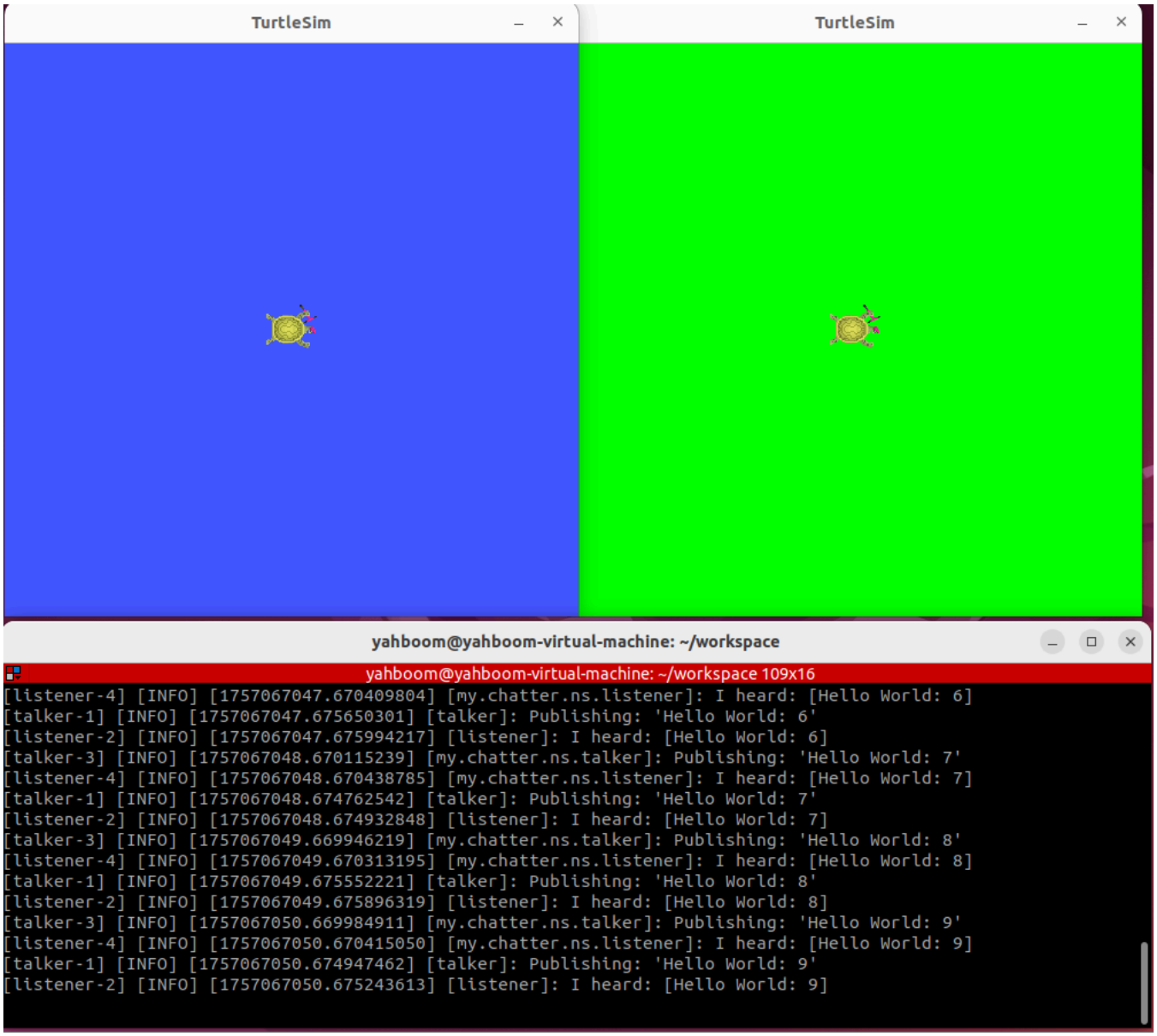


8.4. Running the program

- Refresh the environment variables and then run

```
ros2 launch learn_launch complex_launch.yaml
```

- As expected, two little turtles will appear, and the terminal will print log information



- Start the keyboard control node and add a namespace

```
ros2 run turtlesim turtle_teleop_key --ros-args -r __ns:=/turtlesim1
```

Use the keyboard control to start turtle 1 running; turtle 2 will completely imitate the behavior of turtle 1